

Cell 11 A

Should the graph start traversing all 194 documents?

✗ No.

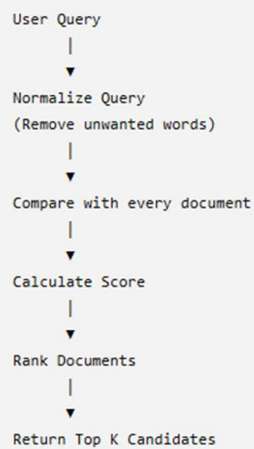
That would be slow and inefficient.

Instead, Cell 11A answers:

"Out of these 194 documents, which ones are the best starting points?"

Those best starting points are called **Candidate Documents**.

Overall Flow



Part 11

Now calculate score.

</> Python

```
score=  
  
overlap*10+  
  
pagerank*100+  
  
degree*2
```

This is your ranking formula.

This is your ranking formula.

Keyword Match

More matching words

↓

Higher score.

PageRank

More important document

↓

Higher score.

Degree

More graph connections

↓

Higher score.

How the clucation is working

Let's look at your code:

Python

```
score = (  
    overlap * 10 +  
    row["pagerank"] * 100 +  
    row["degree"] * 2  
)
```

Then later:

Python

```
if word in title:  
    score += 20
```

and

Python

```
if word in reference:  
    score += 15
```

So the final score is:

```
Final Score =  
Keyword Overlap Score  
+ PageRank Score  
+ Degree Score  
+ Title Bonus  
+ Reference Bonus
```



How to explain it in your meeting

"After building the Knowledge Graph, we run the PageRank algorithm using NetworkX. Initially, every document is assigned an equal importance score. The algorithm then iteratively updates these scores based on the graph structure: documents that receive links from many other documents, especially from already important ones, gain higher PageRank. Once the values converge, the final PageRank score is stored in each node and later used during candidate ranking."

One important clarification

In your project, PageRank is based only on the graph structure.

It **does not** consider:

- the user's query,
- the document content,
- or semantic similarity.



Complete Flow

User Query



Normalize Query



Remove Stop Words



Compare with every document



Calculate Score



Rank Documents



Take Top K



Pass to Cell118

Notes for your notebook

Rank Documents

- Calculate the score for every matching document.
- Sort documents in descending order based on score.
- Highest score = most relevant document.
- Select only the Top-K documents.
- Pass these candidate documents to Cell 11B for graph traversal.

Meeting explanation

"After computing a relevance score for each matching document, we sort all candidate documents in descending order of score. The highest-ranked documents are considered the best starting points for graph traversal. We then select only the Top-K candidates to reduce the search space and improve retrieval efficiency."

This is exactly what the **ranking** step in Cell 11A is doing.

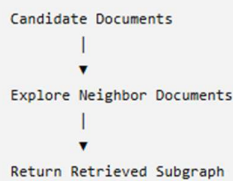
Cell B

Cell 11B – Priority Knowledge Graph Traversal

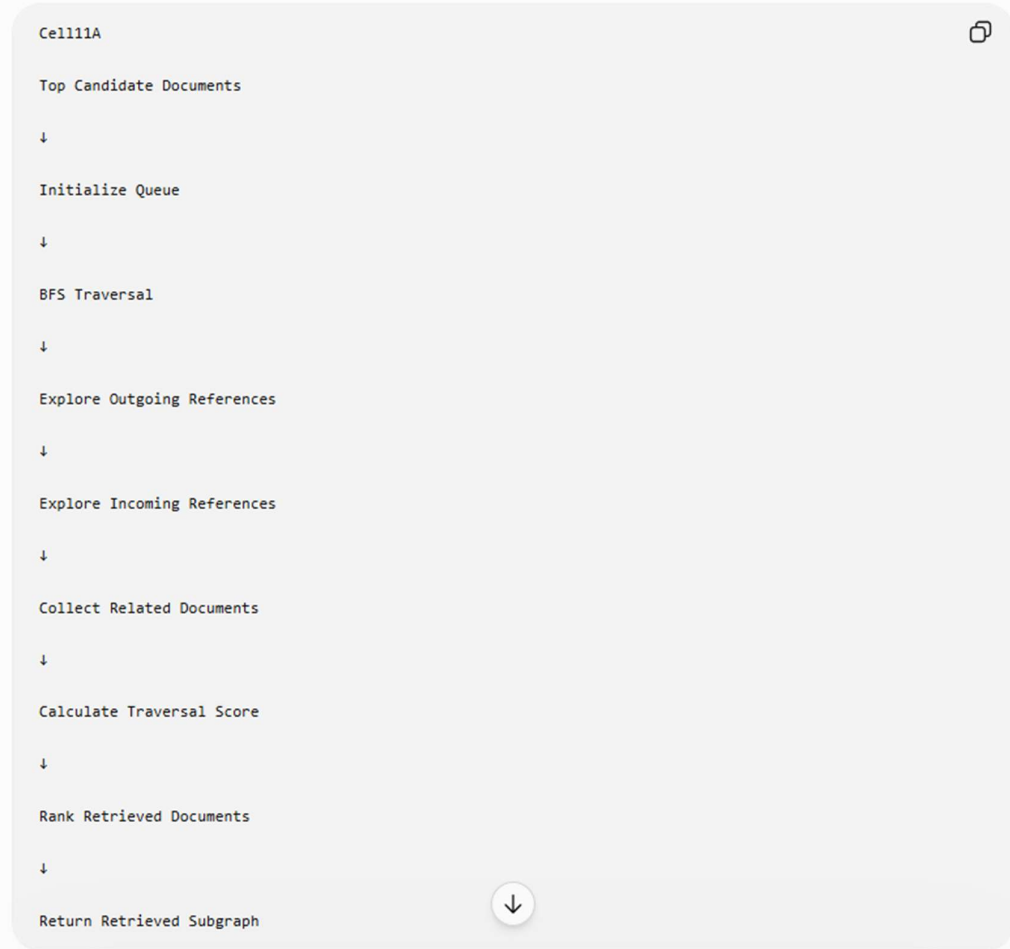
What is this cell doing?

This cell starts from the **Top-K candidate documents** (from Cell 11A) and traverses the Knowledge Graph using **Breadth-First Search (BFS)** to retrieve all related documents.

Simply:



Complete Flow



Notes for your notebook

Cell 11B – Priority Knowledge Graph Traversal

Purpose

- Start from candidate documents.
- Traverse the Knowledge Graph.
- Retrieve connected documents.

Algorithm

- Breadth-First Search (BFS).

Uses

- Queue (deque)

- Visited set
- Outgoing references (successors)
- Incoming references (predecessors)
- Hop limit
- Traversal score

Output

- Ranked retrieved documents (retrieved subgraph).

Meeting Explanation (1 minute)

"Cell 11B performs the graph retrieval stage. Starting from the candidate documents selected in Cell 11A, it performs a Breadth-First Search over the Knowledge Graph. During traversal, it explores both outgoing references and incoming references up to a configurable hop limit. Each retrieved document is assigned a traversal score based on its PageRank, graph degree, and hop distance. Finally, the retrieved documents are ranked and returned as a connected subgraph, which provides richer context for the LLM."

The one sentence you should always remember

Cell 11A answers:

"Which documents best match the user's question?"

Cell 11B answers:

"Which other documents are connected to those documents and should also be retrieved?"

Cell 11C

Excellent! You've now reached the **last part of the Graph Retriever**.

Think of it like this:

- **Cell 11A** → Find the best starting documents.
- **Cell 11B** → Explore the Knowledge Graph.
- **Cell 11C** → Convert the retrieved graph into text that the LLM can understand.

This is an important point:

An LLM cannot understand a NetworkX graph or Python dictionaries. It understands only text (prompts).

That's why Cell 11C exists.

Cell 11C – Knowledge Graph Context Builder

What is this cell doing?

It converts the retrieved graph documents into **one structured text context** that can be passed to the LLM.

Why do we need Cell 11C?

Suppose Cell 11B returns

</> Python



Run

```
[
  {
    "doc_id":35,
    "title":"Fuel System",
    "score":105
  },
  {
    "doc_id":19,
    "title":"Engine",
    "score":92
  }
]
```

Can Azure OpenAI answer using this?

✖ No.

The LLM needs something like

Document 35
Title : Fuel System



Content :
Fuel system controls...

Document 19
Title : Engine

Content :
Engine converts...



This becomes the prompt sent to Azure OpenAI.

Notes for Your Notebook

Cell 11C – Knowledge Graph Context Builder

Purpose

- Convert retrieved graph documents into text.
- Build the LLM prompt.

Input

- Knowledge Graph
- Retrieved Documents

Processing

- Sort by score.
- Remove duplicates.
- Read document content.
- Limit content length.
- Select Top 10 documents.
- Merge into one context string.

Output

- Final context for the LLM

Cell 12 – GraphRAG Retrieval Pipeline

1. What is Cell 12?

Think about the previous cells.

- Cell 11A knows how to find candidate documents.
- Cell 11B knows how to traverse the graph.
- Cell 11C knows how to build graph context.

But if a user asks:

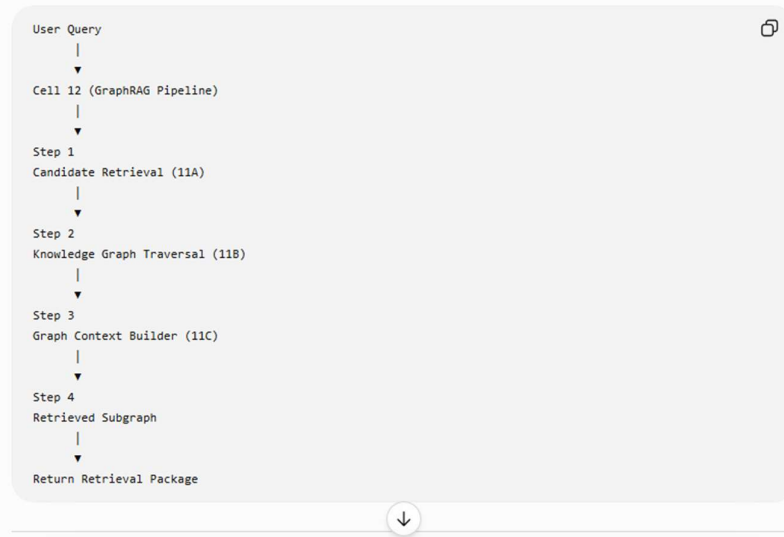
"Explain the Fuel System."

Who calls all these functions in the correct order?

👉 **Cell 12 does.**

It is the **controller (or orchestrator)** of your GraphRAG retrieval pipeline.

Overall Flow



Why do we need Cell 12?

Without Cell 12, the user would have to manually call:

Python

```
retrieve_candidates()

traverse_graph()

build_graph_context()
```

every time.

Instead, the user only calls

Python

```
graphrag.retrieve(query)
```

and Cell 12 automatically executes the complete retrieval pipeline.

Cell 12 acts as the orchestrator of the GraphRAG retrieval pipeline. It doesn't implement a new retrieval algorithm; instead, it coordinates the components built earlier. It first invokes candidate retrieval to identify the most relevant starting documents, then performs Knowledge Graph traversal to collect related documents, builds the graph context from the retrieved information, creates a retrieved subgraph containing only the relevant nodes and edges, and finally returns a complete retrieval package. This package is then used by the answer generation stage.

Cell 14 executes the complete GraphRAG retrieval pipeline. It accepts a user query, initializes the GraphRAG object with the Knowledge Graph and Graph Index, invokes the retrieval pipeline, and receives the retrieval result. The output includes the retrieved documents, their graph relationships, and summary statistics such as the number of retrieved nodes and edges. This step validates that the retrieval pipeline is functioning correctly before passing the retrieved context to the language model."

Notes

Cell 14

Purpose

- Execute the GraphRAG pipeline.
- Test retrieval on a real query.

Input

- User query.

Processing

- Initialize GraphRAG.
- Call retrieve().
- Display retrieved documents.
- Display retrieved relationships.
- Display retrieval statistics.

Output

- Retrieved documents.
- Retrieved subgraph.
- Retrieval statistics.

This is actually a **very good slide**. If you explain it properly, it will become the strongest part of your presentation because this is where your **GraphRAG retrieval logic** is shown.

The best way is **don't explain every box**. Explain it as **three stages**:

- **11A → Find the starting documents**

- 11B → Expand using the Knowledge Graph
 - 11C → Build the context for the LLM
-

Presentation Script (2–3 minutes)

Introduction

"This slide explains how the GraphRAG retriever works. The retrieval process is divided into three stages: Candidate Retrieval, Graph Traversal, and Context Builder. Together, these three stages retrieve the most relevant knowledge from the Knowledge Graph before sending it to the LLM."

Cell 11A – Candidate Retrieval

(Point to the green section.)

Say:

"The first stage is Candidate Retrieval. When a user enters a technical question, the query is first cleaned by removing stop words, converting it to lowercase, and extracting the important keywords."

Example:

User Question

What is the fuel system in Rolls-Royce engine?

↓

Keywords

fuel
system
rolls
royce
engine

Continue:

"These keywords are then compared with every XML document stored in the graph index."

Explain the Scoring

Point to the scoring box.

Say:

"Each document receives a retrieval score based on multiple factors."

Explain one by one:

- Keyword overlap
- PageRank
- Degree
- Title match
- Reference match

Then say:

"After calculating the score, all documents are ranked from highest to lowest. Finally, the Top-K documents are selected as the starting points, also called seed nodes."

Point to:

Top-K Candidate Documents

Cell 11B – Graph Traversal

(Point to the blue section.)

Say:

"The second stage is Graph Traversal. Instead of sending only the Top-K documents to the LLM, we use the Knowledge Graph to retrieve additional related documents."

Continue:

"Starting from the seed nodes, we perform Breadth-First Search, or BFS."

Point to the graph.

Explain:

"The traversal explores both outgoing and incoming relationships."

Explain the colors:

- Dark Blue → Seed Nodes

- Light Blue → One-Hop Documents
- Yellow → Two-Hop Documents

Continue:

"The traversal continues up to two hops. This allows us to retrieve documents that are directly or indirectly connected to the original query."

Then conclude:

"The output of this stage is a retrieved subgraph containing the relevant documents and their relationships."

● Cell 11C – Context Builder

(Point to the purple section.)

Say:

"The final stage is the Context Builder. The retrieved documents are first sorted according to their retrieval score."

Explain each step:

- Remove duplicate documents
- Read XML content
- Trim long documents
- Select Top-N documents
- Combine everything into one context

Continue:

"The final output is a structured Graph Context containing document content, metadata, and relationships."

Then point to:

Final Graph Context

Say:

"This Graph Context is sent to the LLM instead of sending the complete XML repository."

Bottom Flow

(Point to the bottom.)

Explain:

"The retriever produces three outputs."

Point one by one:

Retrieved Nodes

"These are the important XML documents."



Retrieved Relationships

"These are the document connections discovered during graph traversal."



Final Graph Context

"This is the final structured information that is provided to the LLM."

Conclusion

Point to the orange box.

Say:

"Finally, the Graph Context is passed to the LLM, which generates the answer using only the retrieved knowledge instead of the entire XML dataset."

30-Second Summary

If your manager asks:

"Explain your retriever in 30 seconds."

You can answer:

"Our retriever works in three stages. First, Candidate Retrieval ranks all XML documents based on keyword matching, PageRank, degree, title, and reference matching, selecting the Top-K seed documents. Second, Graph Traversal expands these seed documents using Breadth-First Search to retrieve connected documents within two hops. Finally, the Context Builder ranks the retrieved documents, removes duplicates, extracts the relevant XML content, and builds a structured Graph Context, which is then passed to the LLM for answer generation."

